Section 1: Task 1

Difference Between Hot Reload and Hot Restart

Hot Reload: Hot Reload updates the running Flutter application with changes made to the source code without restarting the entire application. It usually preserves the current application state. Developers should use Hot Reload when changing the user interface, text, colors, or widget layout and they want to see the changes quickly.

Hot Restart: Hot Restart restarts the Flutter application and runs the updated code from the beginning. It resets the current application state, but generally does not require rebuilding the entire application from scratch. Developers should use Hot Restart when changes affect initialization logic or when Hot Reload does not apply the changes correctly.

Conclusion: Hot Reload is faster and is useful for small code changes while developing. Hot Restart is useful when the application needs to restart and reset its state.

Section 2: Task 2

Why Applications Should Not Connect as root

Using the MySQL root account for a normal application backend is poor security practice because root has extensive administrative privileges. If an application is compromised or contains a database credential leak, an attacker could potentially modify or delete databases, create users, or change server settings.

The principle of least privilege means giving each application only the permissions it needs. A dedicated application user reduces the damage that can occur if its credentials are exposed.

The following commands create a dedicated user and grant permissions only on the school database. Replace the example password with a strong, unique password before running the commands.

```
CREATE USER 'school_app'@'localhost'
IDENTIFIED BY 'ReplaceWithAStrongUniquePassword!';

GRANT SELECT, INSERT, UPDATE, DELETE
ON school.*
TO 'school_app'@'localhost';
```

SHOW GRANTS FOR 'school_app'@'localhost';

The `CREATE USER` command creates the account. The `GRANT` command permits reading, adding, updating, and deleting records within the school database without giving the account global administrative privileges. `SHOW GRANTS` verifies the permissions assigned.

The application should use this dedicated account rather than root. Its password should be stored securely outside the source code, and the account should receive additional permissions only when required.

Section 3: Task 3 - Python Virtual Environment

Objective

The objective of this task was to create a Python project directory, set up an isolated virtual environment, install the requests library, verify the installation, and export the environment configuration to a requirements.txt file.

Procedure and Commands Used

I opened the integrated terminal in Visual Studio Code and used PowerShell to create the project directory and virtual environment.

cd $HOME

mkdir python_setup_lab

cd python_setup_lab

python -m venv venv

.\venv\Scripts\Activate.ps1

python -m pip install requests

python -m pip list

python -m pip freeze > requirements.txt

Get-Content requirements.txt

Explanation

The `mkdir` command creates the project directory named `python_setup_lab`. The `cd` command navigates into the directory. The `python -m venv venv` command creates an isolated virtual environment named `venv`, while the activation command enables that environment in PowerShell.

The command `python -m pip install requests` installs the requests library. The command `python -m pip list` displays the installed packages so that I can verify the installation.

Finally, `python -m pip freeze > requirements.txt` exports the installed package names and versions into a requirements file. This file can help recreate the same Python dependencies in another environment.

Result

The task is complete when the virtual environment is active, the requests library appears in the package list, and the requirements.txt file contains the installed package versions.

Evidence: Insert a screenshot of the actual terminal output showing the active virtual environment, package verification, and requirements.txt contents.

Section 4: Task 4 - VS Code Workspace Screenshot

Objective

The objective of this task was to configure Visual Studio Code for Flutter, Dart, Python, and MySQL development, and to select the Python interpreter located inside the project's virtual environment.

Extensions Installed

The following extensions were installed or verified in Visual Studio Code:

1. Flutter
2. Dart
3. Python
4. Pylance
5. MySQL

Python Interpreter Configuration

I opened the `python_setup_lab` project folder in Visual Studio Code and selected the Python interpreter located at `venv\Scripts\python.exe`. This configures the workspace to use the isolated Python environment created for Task 3.

I also opened the integrated terminal and activated the virtual environment. The `(venv)` prompt indicates that the environment is active in the terminal.

Result

The workspace is configured to use the project's Python virtual environment, and the required extensions are installed or verified.

Evidence: Insert the required full-screen screenshot showing the Extensions panel, active `(venv)` terminal prompt, and selected Python interpreter in the status bar.